Connection Pooling (HikariCP) in Spring Boot

1. The Problem Without Connection Pooling

When a Spring Boot application needs to talk to a database, it must create a **database connection**.

Creating a DB connection is **expensive** because it involves:

1. TCP connection establishment
2. Authentication with the DB
3. Session creation
4. Resource allocation

If every request creates a new connection:

```
Client Request → Create DB Connection → Execute Query → Close Connection
```

This causes:

- High latency
- Database overload
- Poor scalability

For example, if your healthcare SaaS system receives **1000 requests/sec**, creating 1000 DB connections/sec would overwhelm the database.

---

2. What Connection Pooling Does

A **connection pool** maintains a **set of reusable database connections**.

Instead of creating a new connection each time, the application **borrows a connection from the pool**.

```
Application Request
        ↓
Borrow Connection from Pool
        ↓
Execute Query
        ↓
Return Connection to Pool
```

So connections are **reused**, not recreated.

Example pool:

```
Connection Pool
```

```
[Conn1] [Conn2] [Conn3] [Conn4] [Conn5]
```

When a request comes:

```
Thread → borrow Conn2
Thread → return Conn2
```

---

3. Why HikariCP?

HikariCP is the **default connection pool in Spring Boot**.

Reasons:

- Extremely fast
- Low latency
- Minimal overhead
- Highly optimized
- Better than older pools like:
  - Apache DBCP
  - C3P0
  - Tomcat Pool

Spring Boot automatically configures it when you use:

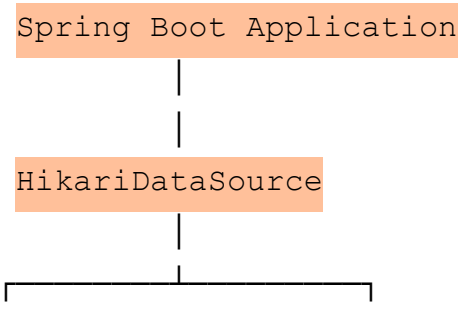
```
spring-boot-starter-data-jpa
```

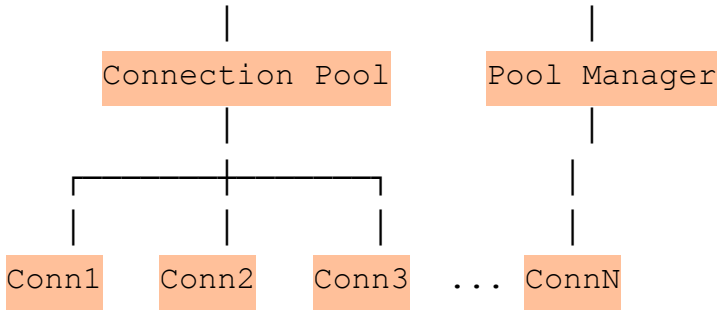
or

```
spring-boot-starter-jdbc
```

---

4. Internal Architecture





Components:

1. HikariDataSource

Main entry point used by Spring.

Spring injects it into:

- JdbcTemplate
- JPA
- Hibernate

---

2. Connection Pool

Stores multiple DB connections.

Example:

```
Pool size = 10
```

```
[Conn1][Conn2][Conn3][Conn4][Conn5]
[Conn6][Conn7][Conn8][Conn9][Conn10]
```

---

3. Pool Manager

Responsible for:

- Creating connections
- Monitoring connections
- Replacing dead connections
- Enforcing timeouts

---

5. Request Flow

When a query runs:

```
@Service
    ↓
Repository
    ↓
Hibernate / JdbcTemplate
    ↓
HikariDataSource
    ↓
Connection Pool
    ↓
Database
```

Detailed flow:

```
Thread requests connection
          ↓
Pool checks available connection
          ↓
If available → return connection
          ↓
Execute SQL
          ↓
Connection returned to pool
```

If pool exhausted:

```
Thread waits until connection is available
```

---

6. Spring Boot Default Configuration

Spring Boot auto-configures HikariCP.

Example properties:

```
spring.datasource.url=jdbc:mysql://localhost:3306/app
spring.datasource.username=root
spring.datasource.password=secret
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

Spring Boot automatically creates:

`HikariDataSource`

---

7. HikariCP Configuration

Example:

```
spring.datasource.hikari.maximum-pool-size=20
spring.datasource.hikari.minimum-idle=5
spring.datasource.hikari.idle-timeout=300000
spring.datasource.hikari.max-lifetime=1800000
spring.datasource.hikari.connection-timeout=30000
```

Explanation:

Property	Meaning
maximum-pool-size	Maximum DB connections
minimum-idle	Minimum idle connections kept
idle-timeout	Close idle connection after this
max-lifetime	Maximum life of connection
connection-timeout	Wait time for connection

---

8. Example Spring Boot Flow

Example repository:

```
@Repository
public class PatientRepository {

    @Autowired
    private JdbcTemplate jdbcTemplate;

    public List<String> findPatients() {
        return jdbcTemplate.queryForList(
            "SELECT name FROM patients",
```

```
                String.class
        );
    }
}
```

Flow:

```
PatientRepository
      ↓
JdbcTemplate
      ↓
DataSource
      ↓
HikariCP Pool
      ↓
Borrow Connection
      ↓
Execute Query
      ↓
Return Connection
```

---

9. Monitoring HikariCP with Actuator

Since you were asking about **Spring Boot Actuator metrics earlier**, Hikari integrates with **Micrometer**.

Enable:

```
management.endpoints.web.exposure.include=health,metrics
```

Metrics available:

```
hikaricp.connections
hikaricp.connections.active
hikaricp.connections.idle
hikaricp.connections.pending
```

Example:

```
/actuator/metrics/hikaricp.connections.active
```

Shows number of active connections.

---

10. Real Production Example

Suppose your healthcare SaaS receives:

```
2000 requests/sec
```

You configure:

```
maxPoolSize = 30
```

Flow:

```
2000 requests
      ↓
30 DB connections
      ↓
Requests share connections efficiently
```

Without pooling:

```
2000 DB connections
```

Database would crash.

---

11. Common Mistakes

Too many connections

Bad:

```
maximum-pool-size = 200
```

Database may not support that.

Too small pool

Bad:

```
maximum-pool-size = 2
```

Threads will wait.

Not closing connections

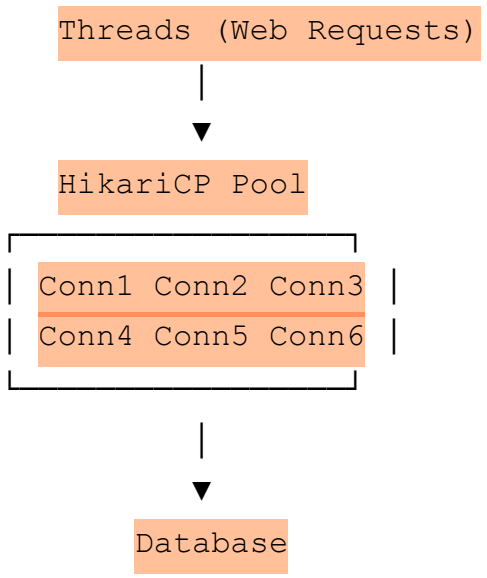
Spring solves this automatically using:

```
@Transactional
JdbcTemplate
Hibernate Session
```

Connections are returned automatically.

---

12. Simplified Diagram



💡 **Rule of thumb for pool size**

```
Pool Size ≈ (CPU cores * 2) + effective disk count
```

Example:

```
8 core server → pool size ≈ 16
```

✅ **In one sentence**

HikariCP is a high-performance JDBC connection pool used by Spring Boot to efficiently reuse database connections instead of creating a new one for every request.

If you'd like, I can also explain **HikariCP internals (ConcurrentBag, FastList, pool algorithms)** — which is a **very popular Spring Boot interview topic for senior engineers.**